Todo App — Data Architecture Document

> Attached to: docs/todo-app/user-story.md Type: data-architecture

Data Domain

Single domain: **Task Management** Owner: backend service No cross-domain data sharing in this phase.

Storage Strategy

Environment	Database	Rationale
Production	PostgreSQL	Persistent, ACID-compliant, supports UUID natively
Local / Test	H2 (in-memory)	Zero-config, fast test execution, Flyway compatible

Schema managed exclusively via Flyway migrations — no manual DDL changes. All migrations versioned and stored in `src/main/resources/db/migration/` .

Data Model

Table: `tasks`

Column	Type	Nullable	Constraints	Notes
id	UUID	no	PRIMARY KEY	Server-generated via `UUID.randomUUID()`
title	VARCHAR(255)	no	NOT NULL	Required on creation
description	VARCHAR(1000)	yes	—	Optional free text
status	VARCHAR(20)	no	NOT NULL, DEFAULT 'ACTIVE'	Enum: `ACTIVE` , `COMPLETED`
priority	VARCHAR(10)	no	NOT NULL, DEFAULT 'MEDIUM'	Enum: `HIGH` , `MEDIUM` , `LOW`
labels	TEXT[]	yes	—	PostgreSQL array; H2 uses JSON column

Column	Type	Nullable	Constraints	Notes
created_at	TIMESTAMP	no	NOT NULL	Set on insert, never updated
updated_at	TIMESTAMP	no	NOT NULL	Updated on every write

Indexes

Index	Columns	Purpose
`idx_tasks_status`	`status`	Filter by status
`idx_tasks_priority`	`priority`	Filter by priority
`idx_tasks_created_at`	`created_at`	Default sort

Data Flow

Write Path (Create / Update / Delete)

```
HTTP Request → TaskController → TaskService → TaskRepository → PostgreSQL
```

- `createdAt` set once on insert by service layer — never overwritten
- `updatedAt` set on every write by service layer
- No event publishing in this phase

Read Path (List / Get)

```
HTTP Request → TaskController → TaskService → TaskRepository → PostgreSQL
                                                    ↓
                                          Filter + Sort applied at DB level
                                                    ↓
                                          TaskResponse mapped in service
```

- Filtering applied via JPA query methods — pushed to DB, not in-memory
- Sorting applied via `Sort` object passed to repository
- Empty result set returns `[]` — never `null`

Migration Strategy

- Tool: Flyway
- Location: `src/main/resources/db/migration/`

- Naming: `V{N}__{description}.sql` (e.g. `V1__create_tasks_table.sql`)
- Migrations run automatically on application startup
- Never modify an existing migration — always add a new versioned file

V1 — Initial Schema

Creates `tasks` table with all columns, defaults, and indexes.

Data Constraints

- `id` is always server-generated — never accepted from client input
- `createdAt` is immutable — set once, never updated by any operation
- `status` and `priority` are validated against their enum values before persistence
- `title` max 255 chars enforced at both API validation and DB column level
- `description` max 1000 chars enforced at both API validation and DB column level
- Labels stored as array — empty array and null treated equivalently

Test Data Strategy

- H2 in-memory database used for all unit and integration tests
- Flyway runs same migrations against H2 — schema parity guaranteed
- Test data seeded via `@Sql` annotations or `DataLoader` beans per test class
- No shared mutable state between tests